



Case2



Guide de Code Review — Feuille à remplir

| Objectif : utiliser cette feuille comme un *fil conducteur* pendant l'exercice.

| Remplis les zones "👉 Réponse" au fur et à mesure.



Étape 1 — Cadrage (Top-Down / Haut Niveau)

1) Intention Métier

Question : À quoi sert ce fichier ? Quel problème résout-il ?

👉 **Réponse :**

- Contexte / produit : Chatbot pour entreprise avec Agent workflow
- Problème utilisateur / besoin : executer des recherches dans DB, avoir des infos sur les data
- Entrées principales : requête utilisateur format json contenant 1-query, 2-identifiant de l'utilisateur
- Sorties principales : réponse du chatbot string contenant du json
- Hypothèses / contraintes :

2) Architecture Globale

Question : Comment le code est-il découpé ? (classes, services, endpoints, fonctions utilitaires)

👉 **Réponse :**

- Composants / modules : 1-Classe EntrepriseChatbotAgent → contient le vector_store, modèle mistral, et un historique de discussion global. 2-Classe ToolExecutionService → Boite a outils du LLM, peut effectuer des

requetes SQL a la BDD. 3-app et endpoint FastAPI "/chat" → reçoit la query de l'utilisateur au LLM 4-Logger → logs en cas d'erreur

- Données qui circulent (schéma rapide) :

query utilisateur json → user_query, tenant_id → intent (str), context_docs (list(str)), user_query → intent (str), content_str(str), user_query → full_user_input (str) → chat_history (list(str)) → response () → message_obj(str) → assistant_reply (str)

3) Périmètre & Dépendances

Question : Quels sont les modules externes importés et les flux d'entrées/sorties ?

 **Réponse :**

- Dépendances externes (librairies / services) :
 - json, logging, typing, fastApi, pydantic
 - Dépendances internes (fichiers / packages du repo) :
 - aucune
 - I/O (réseau, DB, fichiers, cache, queue, etc.) :
 - requête HTTP query utilisateur / output au llm, database sql query
 - Zones sensibles (auth, secrets, PII, permissions) :
 - API Key laissée dans le code !!! 'MISTRAL_API_KEY = "sk-mistral-live-prod-secret-998877"'
-

Étape 2 — Inspection par les 4 Piliers (Scan ligne par ligne)

| Pour chaque point, note **ce que tu observes** et **les anomalies**.

| (Tu peux référencer des lignes/fonctions : `foo()` , `MyClass.method` , "ligne ~120", etc.)

Pilier 1 — Logique & Concurrency

Checklist mentale :

- Le flux d'exécution est-il correct ?
- Async / threads / événements : pas de blocage, pas de race conditions, pas d'await oubliés ?

Notes / anomalies :

- Aucune fonction async alors que pour optimiser on pourrait probablement paralléliser certaines fonctions, notamment :
`chatbot.handle_user_message(user_query, tenant_id)`. Actuellement code bloquant les requêtes de tous les autres utilisateurs
 - `classify_user_intent` devrait être async car appel à un llm
 - `retrieve_context_documents` devrait être async
-

Pilier 2 — Gestion des Ressources & Mémoire

Checklist mentale :

- Pas de fuite d'état global, objets bien isolés ?
- Connexions / fichiers / sockets : cycle de vie propre (ouverture/fermeture garanties) ?

Notes / anomalies :

- `self.chat_history` → partage de mémoire entre tous les utilisateurs !
 - `self.chat_history[-max_history_size:]` → oubli du system prompt !
-

Pilier 3 — Robustesse & Sécurité

Checklist mentale :

- Erreurs / exceptions : traitées correctement (loggées, relancées, pas masquées) ?
- Inputs : validés (null/absent/format), pas d'injection, pas de confiance implicite ?

Notes / anomalies :

- `MISTRAL_API_KEY = "sk-mistral-live-prod-secret-998877"`
- `return self.execute_sql_query(**args)` (pydantic pour vérifier les param du `tool_call`)

- `return {"error": "Unknown tool"} → faire vrai try except`
 - `results = self.vector_store.search(query=query, top_k=5) → pas de param tenant_id utilisé → faille majeure de fuite de donnée`
 - `chat_endpoint(request: Request) → utiliser pydantic pour être sur que la requête est bien formatée`
-

Pilier 4 — Architecture, Design & Performance

Checklist mentale :

- SRP : une seule responsabilité par fonction/classe ?
- Traitements lourds inutiles (boucles, N+1, requêtes multiples, allocations) ?
- Nommage clair, pas ambigu ?

👉 Notes / anomalies :

- `I64 → intent sert a rien`
 - `DEFAULT_MODEL = "mistral-large-latest"`
 - `classify_user_intent → utilise mistral large pour un truc a faible performance`
 - `model=DEFAULT_MODEL, utiliser modèle adapté`
-

📝 Étape 3 — Feedback Structuré (SÉVÉRITÉ / IMPACT / SOLUTION)

A) Rappel des tags de sévérité

- **[BLOCKER]** : crash fatal, bug logique critique, faille de sécurité majeure
 - **[MAJOR]** : perf sous charge, fuite de ressources, mauvaise architecture
 - **[MINOR]** : lisibilité, typage manquant, refactoring local
 - **[NIT]** : cosmétique / style
-

B) Commentaires de review (à remplir)

Copie-colle ce bloc autant de fois que nécessaire :

- `[TAG_SÉVÉRITÉ] Emplacement / Fonction :`
 - Problème :

- Impact (en prod) :
- Solution (proposition concrète, idéalement avec pseudo-code / code) :

Commentaire 1 :

- [MAJOR] :
 - Problème : Aucune fonction async alors que pour optimiser on pourrait probablement paralléliser certaines fonctions, notamment : `chatbot.handle_user_message(user_query, tenant_id)`.
 - Impact : Actuellement code bloquant les requêtes de tous les autres utilisateurs + sous perf / throughput très diminué
 - Solution : utiliser async / await sur `chatbot.handle_user_message`, `retrieve_context_documents`, `classify_user_intent`

Commentaire 2 :

- [BLOCKER] `EnterpriseChatbotAgent`:
 - Problème : historique commun entre tous les utilisateurs / quoi que pas sur car objet chatbot créé à l'appel de l'endpoint
 - Impact : partage de mémoire entre tous les utilisateurs -> data leak et sous perf
 - Solution : faire une mémoire personnalisée jsp comment

Commentaire 3 :

- [BLOCKER] `truncate_memory_sliding_window` :
 - Problème : `self.chat_history[-max_history_size:]` → oubli du system prompt !
 - Impact : le bot perd sa mémoire de sa fonction de base + peut se faire jailbreak
 - Solution : rajouter le message d'index 0 à part

Commentaire 4 :

- [BLOCKER] `MISTRAL_API_KEY = "sk-mistral-live-prod-secret-998877"`:
 - Problème : clé codée en dur
 - Impact : fuite de donnée
 - Solution : utiliser un secret management -> `os.environ[]`

Commentaire 5 :

- [BLOCKER] `dispatch_tool`:
 - Problème : `self.execute_sql_query(**args)` sans pydantic pour vérifier si les args n'ont pas été hallucinés

- Impact : potentiellement appel avec des mauvais paramètres -> crash
- Solution : utiliser pydantic pour vérifier l'extraction dans args

Commentaire 6 :

- [MAJOR] dispatch_tool:
 - Problème : pas de try / except + logging
 - Impact : pas d'info sur les bugs potentiels + propagation d'erreur
 - Solution : rajouter try except et log + description du fail. A faire sur toutes les fonctions ? Je sais pas, est ce que doit systématiquement y'avoir un try/except sur toutes les fonctions ou bien une méthode pour savoir quand le faire?

Commentaire 7 :

- [BLOCKER] : retrieve_context_documents
 - Problème : self.vector_store.search(query=query, top_k=5) → pas de param tenant_id utilisé
 - Impact : faille majeure de fuite de donnée si jamais la vector database contient des fichiers que l'utilisateur n'est pas censé voir !
 - Solution : utiliser le tenant_id dans la query

Commentaire 8 :

- [BLOCKER] : chat_endpoint(request: Request)
 - Problème : json renvoyé par llm sans vérification pydantic
 - Impact : pas sûr que les champs sont corrects -> crash
 - Solution : utiliser pydantic pour être sûr que la requête est bien formatée

Commentaire 9 :

- [MINOR] :
 - Problème : intent sert à rien
 - Impact : code moins propre
 - Solution : enlever ou ajouter une utilité

Commentaire 10 :

- [____] : DEFAULT_MODEL = "mistral-large-latest"
 - Problème : utilisation d'un modèle énorme par défaut, comme dans classify_user_intent → utilise mistral large pour un truc à faible performance
 - Impact : perte de perf pour des tâches qui n'en ont pas besoin

- Solution : soit utiliser un llm pour choisir le modèle en fonction de la query, soit personnaliser en fct des fonctions
